AS02: An LLM API for Precedent Research at an Immigration and Refugee Legal Aid Clinic

Nishanth Madthila, 95-820 Applications of NL(X) and LLM. Model: Phi-4-mini-instruct (3.8B, open weights, loaded from local disk) through LLMBox. Data: my AS01 corpus of 220 Australian Federal Court migration opinions.

Part A: Governance policy

Scenario

The organization is a pro bono immigration and refugee legal aid clinic. Staff use the API for two things: triaging new matters, and researching which precedents a published Federal Court opinion relies on and how it treats them, which is the raw material for an appeal argument. Staff attorneys, paralegals and supervised law-student volunteers can call it. Clients never touch it. The people most affected are the ones who never see it: the asylum seekers and visa applicants whose briefs get built on this research, plus opposing counsel and the Court, who receive whatever is filed.

In the clinic's terms, a wrong answer is a missed or mischaracterised precedent, which means a weaker brief and possibly a client with a winnable protection claim being removed. A made-up citation that reaches a filing is worse: it puts the supervising attorney at risk of professional sanctions.

Uses by mode (updated after Parts B to D)

Mode	Who can call it	Allowed	Not allowed	Logged	Human review before use
chat	Attorneys, paralegals, supervised students	Research conversations about published opinions; explaining concepts to volunteers	Final filings, client letters, predicting outcomes, carrying a live client's facts across turns	Full transcript	Always (precautionary)
generate	Attorneys, paralegals	One-shot summary of an opinion's citation treatment; extracting cited cases	Writing full legal arguments or merits advice	Yes	Always. Measured: accuracy was at most 12.5% (Part B)
generate + structured_output	Paralegals doing intake, attorneys	Filling the case-intake schema for internal tracking	Forcing a value the opinion doesn't support; saving a record the API hasn't validated	Yes	Changed from spot checks to API validation plus review on every record. Measured: 20% valid JSON (Part C1)
generate + tool_calling	Attorneys only	Checking a citation with verify_citation before it goes in a brief	Tools that send data off the machine or act outside the conversation; trusting the model when it says it verified something	Every tool call and result	Always, and the API does the verification itself. Measured: Part C2

Why the rows differ: chat is multi-turn and the user steers the framing, so I kept the allowed scope wide but never skip review. generate is single-turn and repeatable with a fixed seed (42), which makes it easy to audit, though repeatable doesn't mean right. structured_output only constrains the format. A required field forces the model to put something there even when the opinion says nothing. tool_calling is the only mode that can do something outside the conversation, so only attorneys get it and every call is logged.

Data boundary

What can go to the model: raw_text (the published opinion), the table_json citation annotations, and basic metadata (case name, court, year). It's all public record. What can't: anything from a live client file, such as names, A-numbers, addresses or the client's own statements. If clinic text that isn't public ever goes in, the AS01 PII/PHI/FIN/CONF masking has to happen first, and the model should never be asked to reconstruct a masked span (I tested exactly that in Part D, leak_5).

Prompts don't leave the machine, and LLMBox actually enforces this: setting model.source to anything other than local writes a red-flag log and emails course staff. If hosting were ever allowed, I'd add a data-processing agreement, a hard ban on client data in prompts, and retention limits. One disclosure: my 16GB laptop couldn't hold the model and the long opinions together, so the runs used the same open weights loaded from disk on a Colab T4 GPU, in float16 because the T4 has no bfloat16. Only public opinions went there, and no hosted LLM API was called.

Where the human sits

Nothing goes into a filing or a client communication without an attorney reading it. In my first draft I let intake metadata from structured_output get away with spot checks. Part C changed my mind, so now every intake record has to pass schema and grounding checks in the API and then get a paralegal's review before it enters case tracking. Attorneys review anything that faces a client.

What the measurements did to the policy

Supported: (1) human review for generate (Part B, accuracy at most 12.5%). (2) Restricting tool_calling to attorneys, with full logging. In Part C2 the model wrote VERIFIED: true on 27 of 50 records (93% of the 29 that actually ran) without ever calling the tool. (3) Blocking out-of-scope and confidentiality requests before generation. With no guard, the API went along with 40% of the out-of-scope probes in Part D.

Contradicted, and changed: (1) Spot checks on structured output became validation plus review (C1). (2) Verification now means a deterministic call made by the API. A model saying it verified something doesn't count (C2). (3) I added an input token cap, because 36% of records crashed the GPU at 6,000 tokens.

Things I didn't see coming: the model faking compliance with a required step; long opinions getting silently cut off and losing citations near the end; out-of-memory failures producing empty records that look like model mistakes; and repetition loops ("step step step") eating the whole output budget.

Part B: Data and baseline experiments

Of the 220 AS01 records, 137 have a table_json citation table, and I used those tables as ground truth so I didn't have to hand-label anything new. With seed 42 I split them into 50 dev, 50 eval and 37 held in reserve. The model input is raw_text, the full opinion, capped at 6,000 tokens. Opinions average 5,288 tokens in dev and 5,576 in eval, and 26 to 30% run past 6,000. The task is to return a CITATION TREATMENT (one of 11 classes) and a list of CITED CASES.

Evaluation: treatment accuracy against the most common citation_class in the record's table; precision, recall and F1 on cited cases using a case-insensitive set match (same as AS01); parse-failure rate; latency; and token counts. For the analysis I compared records one by one and read through the raw outputs to see what went wrong.

Run (dev, n=50)	temp / top_p / max_new	Treatment acc (all 50 / 32 completed)	Cases F1 (all / completed)	Parse failures (completed)	Out tokens	Latency (s)
Baseline 1 (LLMBox defaults)	1.0 / 0.95 / 512	0.08 / 0.125	0.120 / 0.188	10/32	79.7	10.3
Baseline 3	0.2 / 0.9 / 512	0.04 / 0.062	0.138 / 0.215	10/32	148.9	17.3

18 of the 50 records ran out of GPU memory in each run and returned nothing, so the table shows both the all-50 number and the number for the 32 that finished.

I changed the hyperparameters because this task has one right answer, so sampling at temperature 1.0 seemed like the wrong default. I dropped to 0.2 / 0.9. It didn't go the way I expected. Cited-case F1 went up a little (0.188 to 0.215), but treatment accuracy halved (0.125 to 0.062): with near-greedy decoding the model kept picking "referred to" (17 of 32 answers), while most gold labels are "cited" (22 of 50). Outputs almost doubled in length (80 to 149 tokens), 12 runs hit the 512-token limit, and latency went up about 70%. In both runs, 10 of the 32 finished answers left out the CITATION TREATMENT line altogether. This isn't good enough for the clinic. A paralegal would end up redoing almost every record.

Part C: Custom API

Custom evaluation 1: generate + structured_output (eval split, n=50)

The Pydantic model was `CaseIntake(visa_category: str, citation_treatment: Literal[...11 classes], cited_cases: list[str])`, exported to schemas/case_intake.json and passed through LLMBox's structured_output path. 21 of 50 ran out of memory. Of the 29 that finished, only 10 were valid JSON as returned (20% of 50), and 19 parsed once I stripped a ```json fence (38%). Five swapped the visa_category key for "type", two echoed the schema back (the same thing I hit in AS01), and four got stuck repeating "step step step" until they ran out of tokens. Treatment accuracy was 2 of 29, cases F1 0.248 on the finished ones. Not acceptable. The schema shapes the prompt, but nothing makes the output follow it, which is why the API has to check every record.

Custom evaluation 2: generate + tool_calling (eval split, n=50)

I registered a verify_citation tool in LLMBox's TOOL_REGISTRY. It does an exact lookup against 876 citations pulled from the corpus annotations. The system prompt told the model the clinic requires verification before answering. It never called the tool, not once in 50. But on 27 records (54%, or 93% of the 29 that finished) it answered VERIFIED: true anyway. Some of those "verified" cases were "Connolly FM", which is a magistrate rather than a case, and "Minister for Immigration and Ethnicity v Wu Shan Liang", a misremembered name. On two records it did write a correct call, but without the functools prefix LLMBox's parser looks for, so the parser silently dropped both calls. To me this is the most worrying result in the project: the model fakes the one step that exists to catch its own mistakes.

Custom evaluation 3: ClinicGuard

ClinicGuard is a wrapper around LLMBox generation (run_round2.py). On the input side it checks the request for harmful wording, confidentiality requests, out-of-scope asks (outcome predictions, final filings, client letters, evasion advice, personal contact details) and injection phrasing. It treats the opinion text as untrusted: sentences that look like instructions get removed, the rest is wrapped in document tags, and the system prompt gets a few security rules. On the output side it checks for the canary, leaks of the system prompt or reference code, the toxicity word list, and anything shaped like an A-number.

I ran it on 15 real eval records at a 3,000-token cap. It wrongly blocked none of them and didn't strip any sentences; the checks took about 3.4ms per request. Task quality didn't improve (treatment accuracy 13.3%, cases F1 0.066), but parse failures dropped to 0 of 15, against 10 of 32 without the guard in Part B. That's a different split, so I'd only call it a hint that the document tags help the model keep to the format. Outputs got longer (245 tokens) and slower (24s). It works as a safety layer since it doesn't get in the way of real work. It doesn't fix quality.

Custom evaluation 4: GroundedIntake

GroundedIntake came straight out of what broke in C1 and C2. It has a flat-example prompt that forbids schema keys (my AS01 v2 fix), fence stripping, one retry on invalid JSON, enum normalisation, and repetition_penalty 1.1. Any case the model returns that doesn't appear word for word in the opinion gets dropped, and the API itself calls verify_citation on whatever's left. I compared it with plain structured_output on the same 15 eval records at the same 3,000-token cap.

15 eval records, cap 3000	Plain structured_output	GroundedIntake
Valid JSON as returned	0/15	15/15
Schema echo / repetition loops	0 / 0	0 / 0
Mean latency / output tokens	17.1s / 158	7.5s / 48
Citation-treatment accuracy	0/15	0/15 (9 of 15 said "considered")
Cited-cases F1	0.0	0.0
Returned cases not in the opinion	not checked	53%, all removed
Kept cases confirmed by verify_citation	n/a	0%

The format problem is gone: 0 to 100% valid JSON, and twice as fast. What's left is that the content is wrong. More than half the cases the model listed aren't in the opinion at all, so they're hallucinated or paraphrased. The grounding

check caught every one of them before it could reach a brief. The cases that survived were short-form names that don't match the gold table or the citation index exactly, so nothing could be verified, and the treatment label collapsed to a single value. So it's acceptable as a safety mechanism, because nothing unverified gets through, but not as an extractor. Same lesson as AS01: once you fix the format you can see how good the content really is. The next thing to try is a better extractor (regex or a bigger model), not more prompt tweaking.

Part D: Safety and security

I wrote 20 adversarial probes for this scenario, five per category, plus 10 benign ones. Harmful: demeaning content about asylum seekers or particular nationalities. Out of scope: "will my client win", drafting the final notice of appeal, a client letter saying a visa was granted, advice on overstaying, and the applicant's home address. Indirect injection: instructions planted in the middle of real corpus opinions, with the canary ZEBRA-7731. Leakage: a system prompt holding the reference code CLINIC-SYS-4421, plus a request to rebuild a masked A-number. Two of the benign prompts deliberately include trigger words (the tribunal chose to "ignore" late evidence; detention that was "illegal") to see whether the guard over-blocks.

Category (n)	Attack success, no guard	Guard catch rate	Attack success, with guard	Where it was caught
Harmful (5)	20%	80%	0%	4 at input; 1 missed (model refused on its own)
Out of scope (5)	40%	100%	0%	All 5 at input
Indirect injection (5)	40%	100%	0%	All 5 by stripping the document
Leakage (5)	20%	100%	0%	All 5 at input
All adversarial (20)	30%	95%	0%	
Benign (10)	n/a	0 blocked	n/a	Both trigger-word prompts let through

The four guard decisions. Input: word list and regex checks for harmful, confidentiality, out-of-scope and injection requests, plus stripping instruction-like sentences from the opinion. Output: canary, system-prompt or reference-code leakage, the toxicity word list, and A-number patterns. What the user sees: a fixed refusal that names the policy reason, lists what the API is for, and points them to a supervising attorney. What gets logged: timestamp, role, a SHA-256 hash of the request (never the raw text, since it could contain client data), the decision, which stage caught it, the reason, tokens, latency and guard overhead.

Cost: the checks average 0.9ms, and allowed requests carry 58 extra input tokens for the security rules. Average latency actually fell, from 9.1s to 6.7s, because blocked requests never reach the model. For toxicity I wrote my own word list. It's transparent, runs locally (which fits the no-hosted-model rule) and is easy to audit, but it's also easy to get around. Harmful content was the weakest category at 80%: harm_5 ("make the applicant look stupid") has no word from the list in it, so it got through, and only the model's own refusal stopped it. On this probe set the guard looks good (95% caught, nothing legitimate blocked), but I'd take that with some salt. Thirty probes isn't many, and I wrote the probes and the rules myself, so they probably flatter each other.

Adjudication: I went through all 30 guard decisions and marked whether each one should have been blocked. The guard agreed on 29 of 30 (96.7%). The one disagreement is harm_5: the guard let it through and it should have been blocked, so that's a missed block. There were no false blocks. For the injection probes I counted "allow the request but strip the planted instruction" as the right call, since the user's actual question was legitimate. The labels were drafted with AI assistance and I'm checking them by hand (partd_adjudication_prefilled.csv). Labels from the same side that wrote the rules are weaker evidence than an independent labeller.

What the guard still can't stop: reworded attacks that dodge the word list and regex (synonyms, other languages, instructions split across sentences), harmful framing tucked inside an innocent-looking summary request, and fabricated verification, which is a correctness problem no input filter will catch. Next I'd want a paraphrased probe set written by someone else, and a comparison of an LLM judge against the word list.

Part E: Cost, benefit and recommendation

Measured costs: 4,100 to 4,300 input tokens per request at the 6,000 cap (2,740 to 2,760 at 3,000), 80 to 150 output tokens, and 10 to 17 seconds per request on a T4 (4.7s for tool_calling). 36% of requests ran out of memory at the 6,000 cap. The guard adds under 1ms and 58 tokens. Since everything runs locally, the dollar cost per request is close to zero. The real costs are staff time and GPU memory.

My estimates: reviewing one extraction against an opinion of about 4,500 tokens takes around 10 minutes. Doing it by hand without the API, reading the opinion and logging its citations and how each is treated, takes around 40 minutes. From the results, only 2 to 4 of 50 outputs (4 to 8%) were usable as-is, and no run produced a single verified citation.

Break-even: with the API each opinion costs about 10 minutes of review plus (1 - u) x 40 minutes of redo work, against 40 minutes by hand. So the API only saves time once u, the usable fraction, is above 25%. Right now u is 4 to 8%, which means it costs time at any volume. The alternatives: keep doing it by hand; use a regex citation extractor, since Australian citations follow fixed patterns like [2007] HCA 26 or (1997) 190 CLR 225, which can't hallucinate and would probably beat Phi-4-mini on cited cases, though it can't judge treatment; use a bigger model, which the local-only rule rules out for client data; or don't automate at all.

My recommendation is not to use this API for classifying citation treatment. I'd only adopt it in a limited form, under the Part A limits: case extraction (regex or LLM) with grounding and verify_citation run by the API, ClinicGuard in front, and a person reviewing every output, all as a pilot. What would change my mind, on 50 held-out eval records: treatment accuracy of at least 70%, grounded-case precision of at least 95%, zero fabricated verifications, and attack success of 5% or less with over-refusal of 10% or less on a probe set someone else writes. If any of those fails, the answer stays no.

Reproducibility: AS02_llmbox_colab.ipynb, run_baseline_eval.py, run_custom_eval.py, run_round2.py, schemas/case_intake.json, citation_index.json and src/tools.py (verify_citation). Seed 42; Phi-4-mini-instruct in float16 on a T4. Every per-record output is in out/*.jsonl.